

# Vel Tech Group of Institutions

Name: UMA MAHESHWARI S

Email: vtu28408@veltech.edu.in

Roll no: vtu28408

Phone: null

Branch: VTU

Department: CSE with AIDS

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results



## 2028\_NeoColab\_Design and Analysis of AlgorithmsS1L2

### VTU\_DAA\_Week 4\_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

### Section 1 : Coding

#### 1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

#### ***Input Format***

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

### **Output Format**

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 4

0 3 9 5

3 0 6 4

9 6 0 7

5 4 7 0

Output: Minimum Cost is: 21

Optimal Path: 0 1 2 3 0

### **Answer**

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n, cost[10][10];
```

```

int visited[10], path[10], final_path[10];
int minCost = INT_MAX;

void tsp(int pos, int count, int currCost) {
    if (count == n && cost[pos][0]) {
        int total = currCost + cost[pos][0];
        if (total < minCost) {
            minCost = total;
            for (int i = 0; i < n; i++)
                final_path[i] = path[i];
        }
        return;
    }
    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[pos][i]) {
            visited[i] = 1;
            path[count] = i;

            tsp(i, count + 1, currCost + cost[pos][i]);

            visited[i] = 0;
        }
    }
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    visited[0] = 1;
    path[0] = 0;

    tsp(0, 1, 0);

    printf("Minimum Cost is: %d\n", minCost);
    printf("Optimal Path: ");
    for (int i = 0; i < n; i++)

```

```
printf("%d ", final_path[i]);  
printf("0");  
  
return 0;  
} // You are using GCC
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

### ***Input Format***

The first line contains an integer  $n$ , the number of galleries.

The next  $n$  lines contain  $n$  integers each, representing the adjacency matrix.

The  $i$ th integer on the  $j$ th line represents the cost of traveling from gallery  $i$  to gallery  $j$ .

### ***Output Format***

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

### Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n, cost[10][10];
```

```
int visited[10];
```

```
int minCost = INT_MAX;
```

```
void tsp(int pos, int count, int currCost) {
```

```
    if (count == n && cost[pos][0]) {
```

```
        int total = currCost + cost[pos][0];
```

```
        if (total < minCost)
```

```
            minCost = total;
```

```
        return;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (!visited[i] && cost[pos][i]) {
```

```
            visited[i] = 1;
```

```
            tsp(i, count + 1, currCost + cost[pos][i]);
```

```
            visited[i] = 0;
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf("%d", &cost[i][j]);
```

```
    visited[0] = 1;
```

```
    tsp(0, 1, 0);
```

```
printf("Minimum Cost is: %d", minCost);  
  
return 0;  
} // You are using GCC
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid unnecessary calculations.

#### **Input Format**

The first line contains a single integer  $N$ , representing the number of products.

The next  $N$  lines each contain two integers:  $\text{weight}[i]$  and  $\text{value}[i]$ , where  $\text{weight}[i]$  is the weight of the product and  $\text{value}[i]$  is the value (profit) of the product.

The next line contains a single integer  $W$ , representing the weight capacity of the warehouse.

#### **Output Format**

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

#### **Sample Test Case**

Input: 5  
2 40

3 50  
1 100  
5 95  
3 30  
10

Output: 245

**Answer**

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    int wt[n], val[n];
```

```
    for (int i = 0; i < n; i++)  
        scanf("%d %d", &wt[i], &val[i]);
```

```
    int W;  
    scanf("%d", &W);
```

```
    int dp[1001] = {0};
```

```
    for (int i = 0; i < n; i++) {  
        for (int w = W; w >= wt[i]; w--) {  
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);  
        }  
    }
```

```
    printf("%d", dp[W]);
```

```
    return 0;  
} // You are using GCC
```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity. Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

#### Example

Input:

3

60 10

100 20

120 30

50

Output:

220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

#### ***Input Format***

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.



The last line contains an integer C representing the weight capacity of your knapsack.

### ***Output Format***

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

Refer to the sample output for the formatting specifications

### ***Sample Test Case***

Input: 3

60 10

100 20

120 30

50

Output: 220

### ***Answer***

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    int val[n], wt[n];
```

```
    for (int i = 0; i < n; i++)  
        scanf("%d %d", &val[i], &wt[i]);
```

```
    int W;  
    scanf("%d", &W);
```

```
    int dp[20][60];
```

```
for (int i = 0; i <= n; i++) {  
    for (int w = 0; w <= W; w++) {  
        if (i == 0 || w == 0)  
            dp[i][w] = 0;  
        else if (wt[i-1] <= w)  
            dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]);  
        else  
            dp[i][w] = dp[i-1][w];  
    }  
}  
  
printf("%d", dp[n][W]);  
  
return 0;  
} // You are using GCC
```

**Status :** Correct

**Marks :** 10/10